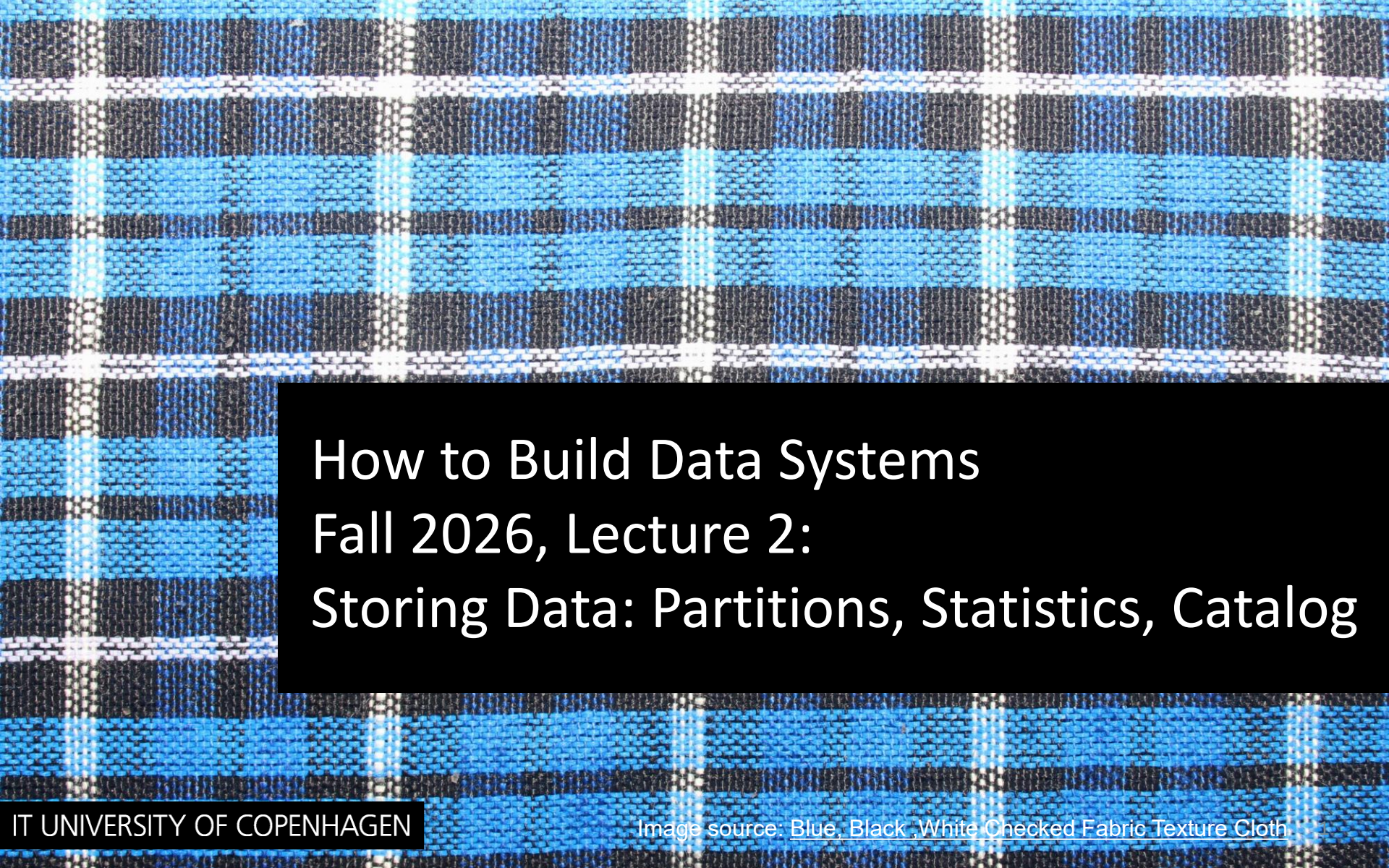


The background of the slide is a close-up texture of a fabric with a blue, black, and white checked pattern. The pattern consists of intersecting horizontal and vertical lines of varying widths, creating a grid-like appearance. The colors are vibrant blue, deep black, and off-white.

How to Build Data Systems

Fall 2026, Lecture 2:

Storing Data: Partitions, Statistics, Catalog

Exercise 1 Recap: Teams



https://ituniversity-my.sharepoint.com/:x:/g/personal/mhent_itu_dk/IQDYeNNXuWPvQLvKJwdfS447Af5K_E8KZ6elgiEucvxQi-E

Exercise 2 Preview

Catalog storage

```
public void createTable(String tableName, List<ColumnSpec> columns) { /* ... */ }
```

```
public void copyFile(String tableName, String csvFilePath) { /* ... */ }
```

```
public List<Object[]> select(String tableName, String columnName,  
                             Comparison comparison, Object constant) { /* ... */ }
```

- CSV parsing
- Data storage
- Partition statistics

Partition pruning

Logging, value encodings & byte order covered
in exercise sheet, not in this lecture

Exercise 2: No Storage Libraries

Not allowed: Parquet, Arrow, or any other storage library

→ Designing the binary format is the point of this week

Allowed: a JSON library for the catalog (e.g., Jackson)

→ Design decision what/how/where to store metadata

Exercise 2 Design Decisions

[docs/storage-design.md](#), before AI starts coding

→ Decided as a team, merged through a reviewed PR

Design decisions: Catalog

- Content
- Storage & format

Design decisions: Data storage

- Binary format (row-wise, column-wise, mixed)
- Partitioning
- Statistics & storage

Agenda

Catalog

- Content
- Storage & format

CSV, the input side

Data

- Binary format (row-wise, column-wise, mixed)
- Partitioning
- Statistics & pruning

From DDLs to Catalog Contents

```
CREATE TABLE trips (city STRING, distance LONG, price DOUBLE);
```



```
public enum ColumnType { STRING, LONG, DOUBLE }
```

```
public record ColumnSpec(String name, ColumnType type) { }
```

```
public void createTable(String tableName, List<ColumnSpec> columns) { /* ... */ }
```

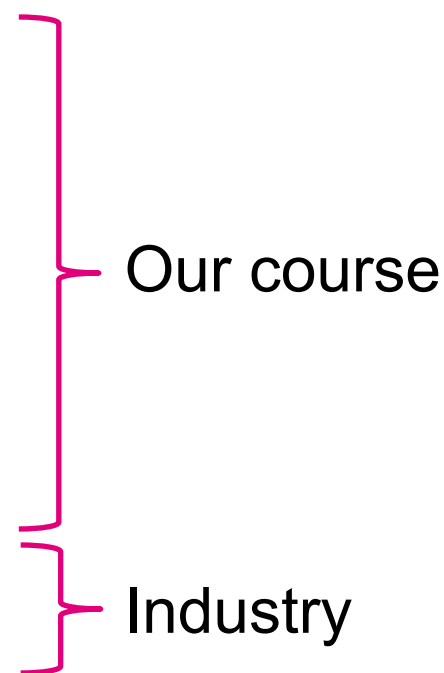


Must be persisted in the catalog
= Table metadata must survive restarting your engine

Catalog Contents

Per table:

- Schema of the table
 - table name
 - list of column names and column types
- List of files
 - list of partitions
 - statistics for partitions/files
- Table versions (later project)
- Users, roles, grants, views, UDFs, encryption keys, connector tokens, ...



Our course

The diagram uses magenta curly braces to group the list items. The first three items (Schema of the table, List of files, and Table versions) are grouped under the label 'Our course'. The last item (Users, roles, grants, views, UDFs, encryption keys, connector tokens, ...) is grouped under the label 'Industry'.

Industry

Catalog Storage

My engine

```
{ Catalog.json
  "tables" : [ {
    "name" : "trips",
    "columns" : [ {
      "name" : "city",
      "type" : "STRING"
    }, {
      "name" : "distance",
      "type" : "LONG"
    }, {
      "name" : "price",
      "type" : "DOUBLE"
    } ],
    "version" : 2,
    "partitions" : [ {
      "offset" : 232,
      "rowCount" : 7,
      "summaries" : [ {
        "min" : "Aalborg",
```



DuckDB

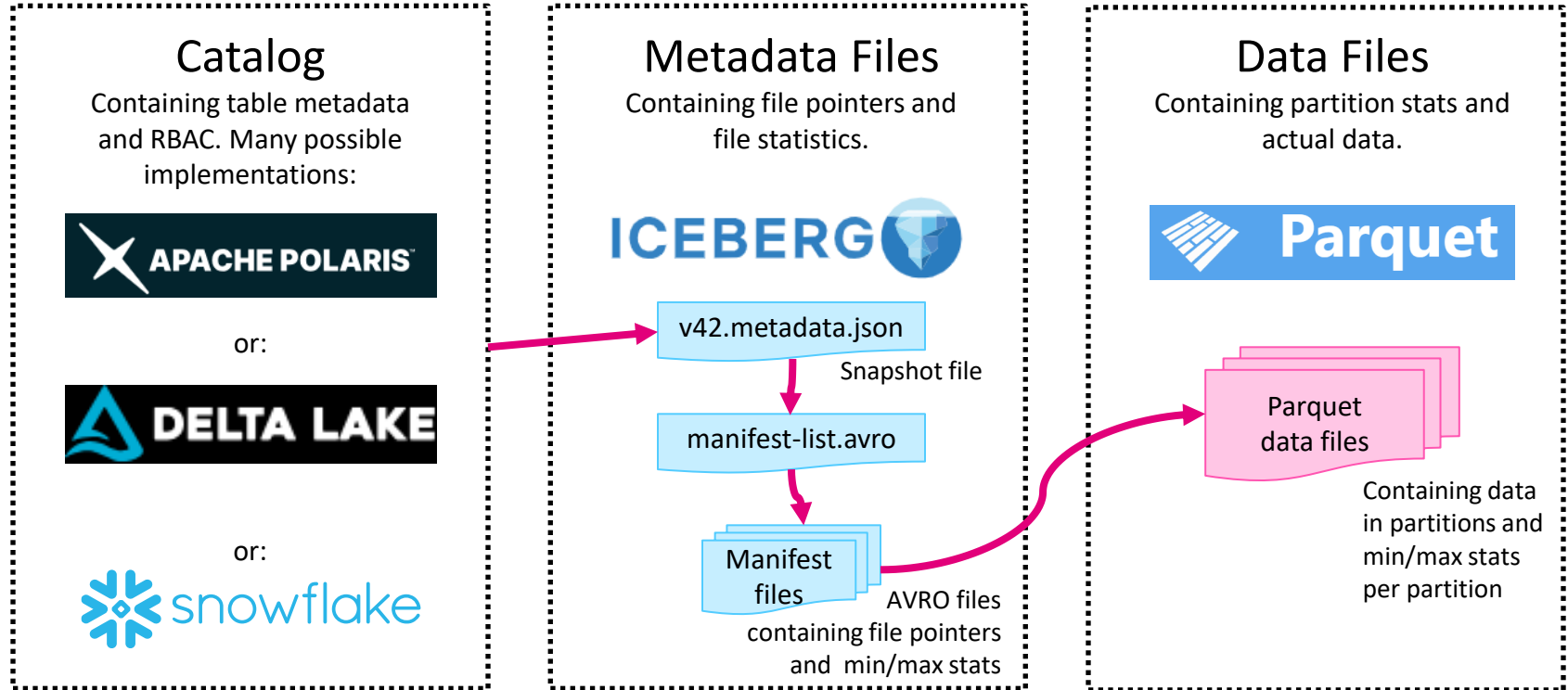
Binary
DB file



All implementations store table metadata, pointers to files & partitions, and partition stats.

(Simplified for Snowflake.)

Metadata in Iceberg + Parquet Data Lakes



Let's discuss

One catalog file vs. one catalog file per table vs. a DB/KV
JSON vs. AVRO/Protobuf vs. your own binary format
Stats inside catalog vs. part of data files vs. a mix

Agenda

Catalog

- Content
- Storage & format

CSV, the input side

Data

- Binary format (row-wise, column-wise, mixed)
- Partitioning
- Statistics & pruning

Exercise 2 Preview

Catalog storage

```
public void createTable(String tableName, List<ColumnSpec> columns) { /* ... */ }
```

```
public void copyFile(String tableName, String csvFilePath) { /* ... */ }
```

```
public List<Object[]> select(String tableName, String columnName,  
                             Comparison comparison, Object constant) { /* ... */ }
```

- **CSV parsing**
- **Data storage**
- **Partition statistics**

Partition pruning

Comma-Separated Values (CSV)

CSV is “everywhere”

- Great tool support
- Human-readable
- Slow to scan
- Underspecified: quoting, types, encodings, headers all vary

Convention in our course

- No header
- Only comma as separator
- No quotes or commas in fields

The Sorry State of the CSV Landscape



Source: [DuckDB's CSV Reader and the Pollock Robustness Benchmark: Into the CSV Abyss – DuckDB](#)

Header or No Header?

With a header: the file is self-describing

- But the header must be validated against the catalog:
names, order, types
- And whether line 1 is a header cannot be sniffed reliably

Without a header: fields match columns by position

- The schema lives in the catalog, where it is already authoritative

→ Our engine: no header!

DuckDB Sniffs the First CSV Line

DuckDB guesses whether line 1 is a header from the types of the first line

- On mixed-type data the guess agrees with our engine:
a **STRING** in line 1 of a **LONG** column looks like a header
- On all-**STRING** files the guess can differ: every line looks like a header
- Adding (**HEADER = false**) to **COPY** pins the behavior down, for example:

```
SELECT * FROM read_csv('trips.csv', header = false);
```

Parsing “Our” CSV

Parsing Steps

1. Java’s `BufferedReader` plus `split()` is enough: our CSV has no quoted fields
2. Convert each field by the column's declared type: `Long.parseLong`, `Double.parseDouble`, or keep the string: our engine has only three types
3. A malformed value fails the whole `COPY`, optionally with file and line number in the error
4. No partial loads: either the file is in the table, or it is not

Agenda

Catalog

- Content
- Storage & format

CSV, the input side

Data

- Binary format (row-wise, column-wise, mixed)
- Partitioning
- Statistics & pruning

Exercise 2 Preview

Catalog storage

```
public void createTable(String tableName, List<ColumnSpec> columns) { /* ... */ }
```

```
public void copyFile(String tableName, String csvFilePath) { /* ... */ }
```

```
public List<Object[]> select(String tableName, String columnName,  
                             Comparison comparison, Object constant) { /* ... */ }
```

- CSV parsing
- **Data storage**
- Partition statistics

Partition pruning

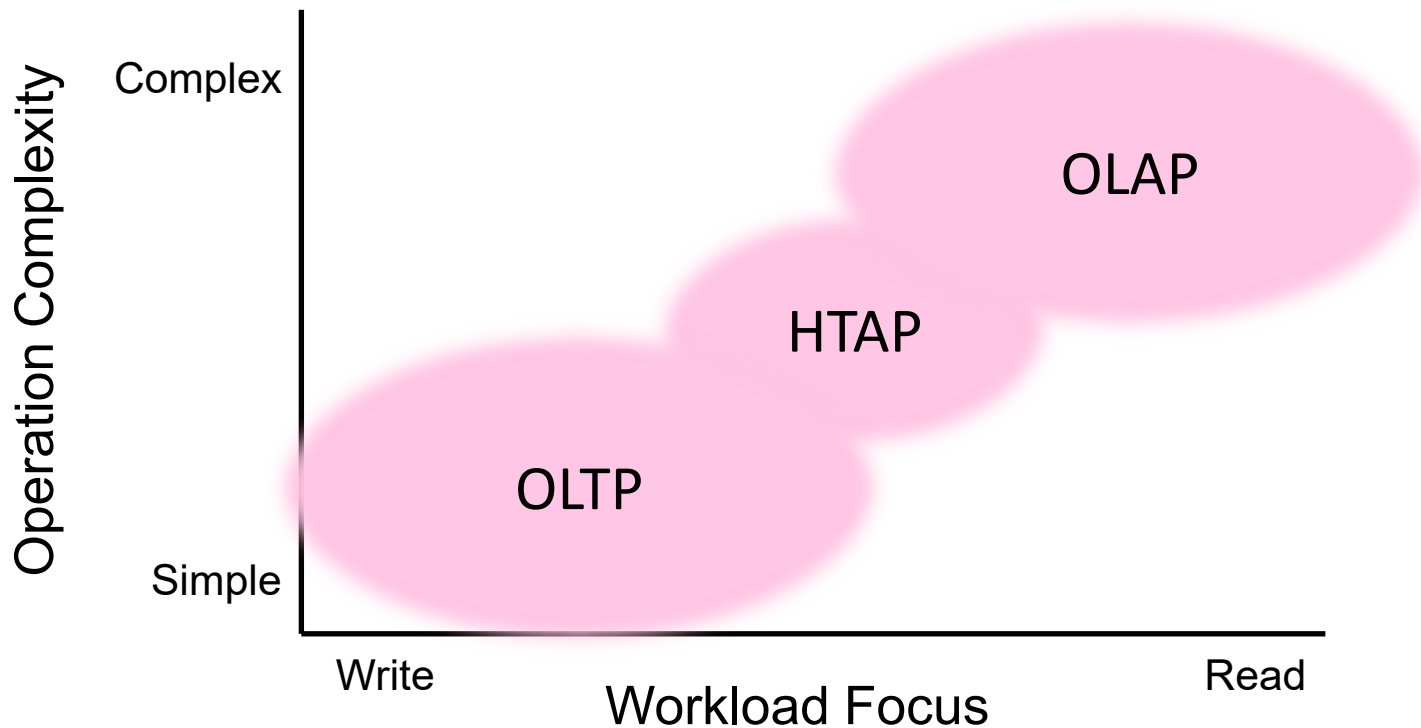
Recap: Introduction to Database Systems, Lecture 11

OLAP vs. OLTP

Storage models:
NSM vs. DSM vs. PAX



Database Workloads



Database Workloads

“Online” = system gives answer immediately
(1980s definition of “online” vs. scheduled jobs on mainframe)

OLTP: Online Transaction Processing

→ Fast operations that only read/update small amount of data each time

OLAP: Online Analytical Processing

→ Complex queries that read a lot of data

HTAP: Hybrid Transaction and Analytical Processing

→ OLTP and OLAP together in the same database

Online Transaction Processing

- Focus on real-time data processing
- High volume of short transactions
- Typically involves frequent insert, update, and delete operations

→ Processes small amounts of data
→ Applications, e.g. Analog App

Example Queries

```
-- price of a tea
select *
from product
where productname = 'Tea';
```

```
-- price of 'fancy' product
-- is unknown
insert into product
  values('Fancy', null);
```

```
-- update user Donna
update user
set email = 'donna@itu.dk',
    password = '***'
where username = 'Donna';
```

Online Analytical Processing

- Focus on querying and reporting
- Complex queries on large datasets
- Typically involves data aggregation and multi-dimensional analysis

→ Processes large amounts of data

→ Data science, ML, visualizations

Example Queries

```
-- how many products did  
-- each customer buy  
select username,  
        count(*) as count  
from purchase  
group by username  
order by count desc;
```

```
-- cumulative sum of sales  
-- per user over time  
select *,  
        sum(price) over  
        (partition by username  
         order by purchasetime)  
        as cum_sales  
from sales  
order by purchasetime;
```

Data in Files

Store data in files by tuples, one tuple after the next

- Header contains metadata like number of tuples, start byte of first tuple, checksums, and more
- Tuples can be of different lengths (for simplicity all same length below)

Purchase (Id, PurchaseTime, ProductName, UserName)

Id	PurchaseTime	ProductName	UserName
1	Feb 11, 9:55	Tea	Martin
2	Feb 12, 10:03	Small	Martin
3	Feb 12, 10:05	Small	Omar
4	Feb 12, 10:06	Large	Omar
5	Feb 19, 9:00	Small	Martin



Purchase.data

Header			
1	Feb 11, 9:55	Tea	Martin
2	Feb 12, 10:03	Small	Martin
3	Feb 12, 10:05	Small	Omar
4	Feb 12, 10:06	Large	Omar
5	Feb 19, 9:00	Small	Martin

N-ary Storage Model (NSM)

DBMS's **storage model** defines how it organizes tuples on disk and in memory

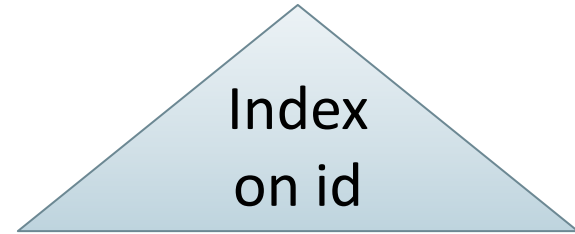
N-ary Storage Model (NSM)

- Store all attributes of tuple in a single file, tuple by tuple
- Known as “row store”

→ Ideal for OLTP workloads where queries are more likely to access individual entities and execute write-heavy workloads

NSM vs. OLTP Example

```
update purchase
set productname = 'Tea'
where id = 2
and username = 'Martin';
```



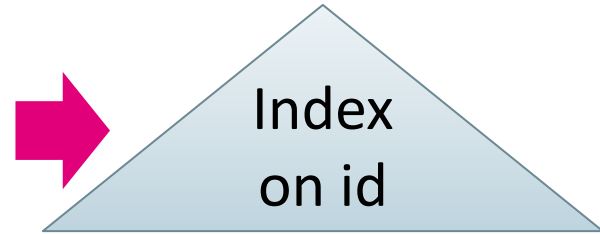
Purchase.data

Header			
1	Feb 11, 9:55	Tea	Martin
2	Feb 12, 10:03	Tea	Martin
3	Feb 12, 10:05	Small	Omar
4	Feb 12, 10:06	Large	Omar
5	Feb 19, 9:00	Small	Martin

- Finding the right tuple to update is fast using the index
 - Comparing username and updating productname is fast because we've loaded the full tuple into memory
- 1 index access, 1 tuple access

NSM vs. OLTP Example

```
insert into purchase  
values (6, '2025-04-09 11:03'::timestamp,  
'Small', 'Martin');
```



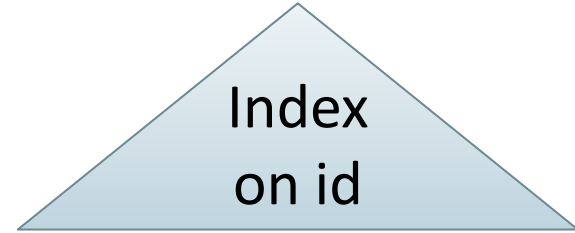
Purchase.data

Header			
1	Feb 11, 9:55	Tea	Martin
2	Feb 12, 10:03	Tea	Martin
3	Feb 12, 10:05	Small	Omar
4	Feb 12, 10:06	Large	Omar
5	Feb 19, 9:00	Small	Martin
6	Apr 09, 11:03	Small	Martin

- Inserting fast: write the tuple to the end of the file
- 1 index update, 1 file write

NSM vs. OLAP – Exercise

```
select productname,  
       count(*) as quantity_sold  
from purchase  
where purchasetime > '2025-02-01'  
and purchasetime < '2025-03-01'  
group by productname;
```



Purchase.data

Header			
1	Feb 11, 9:55	Tea	Martin
2	Feb 12, 10:03	Tea	Martin
3	Feb 12, 10:05	Small	Omar
4	Feb 12, 10:06	Large	Omar
5	Feb 19, 9:00	Small	Martin
6	Apr 09, 11:03	Small	Martin

- Is NSM a good choice for OLAP queries?
- Can you think of some drawbacks of NSM?

NSM vs. OLAP

```
select productname,  
       count(*) as quantity_sold  
from purchase  
where purchasetime > '2025-02-01'  
and purchasetime < '2025-03-01'  
group by productname;
```

- OLAP query reads all tuples, loading all attributes
- 6 tuple accesses
- > 50% attributes uselessly loaded into memory

Purchase.data

Header			
1	Feb 11, 9:55	Tea	Martin
2	Feb 12, 10:03	Tea	Martin
3	Feb 12, 10:05	Small	Omar
4	Feb 12, 10:06	Large	Omar
5	Feb 19, 9:00	Small	Martin
6	Apr 09, 11:03	Small	Martin

Grey = uselessly
loaded data

Advantages

- Fast inserts, updates, and deletes
- Good memory locality for queries that output the entire tuple
- Great for OLTP workloads

Disadvantages

- Not good for scanning large portions of the table and/or a subset of the attributes
- Terrible memory locality for OLAP access patterns
- Not ideal for compression because of multiple value domains within a single file

Decomposition Storage Model (DSM)

Decomposition storage model **decomposes** tuples

- Attributes are stored column by column
 - Known as “column store”
- Ideal for OLAP workloads where read-only queries perform large scans over a subset of the table's attributes
- DBMS is responsible for combining/splitting a tuple's attributes when reading/writing

Purchase (Id, PurchaseTime, ProductName, UserName)

Id	PurchaseTime	ProductName	UserName
1	Feb 11, 9:55	Tea	Martin
2	Feb 12, 10:03	Small	Martin
3	Feb 12, 10:05	Small	Omar
4	Feb 12, 10:06	Large	Omar
5	Feb 19, 9:00	Small	Martin



Purchase.dsm



DSM vs. OLAP Example

```
select productname,  
       count(*) as quantity_sold  
from purchase  
where purchasetime > '2025-02-01'  
and purchasetime < '2025-03-01'  
group by productname;
```

Filter only reads
purchasetime
attributes

Group by only reads
productname attributes

Purchase.dsm

Header					
1	2	3	4	5	
Feb 11, 9:55			Feb 12, 10:03		
Feb 12, 10:05			Feb 12, 10:06		
Feb 19, 9:00					
Tea	Small	Small	Large		
Small					
Martin	Martin		Omar		
Omar	Martin				

DSM vs. OLTP – Exercise

Drawbacks of using DSM for OLTP workloads

`select *` — Reading from multiple pages
`from purchase` or files + tuple reconstruction
`where id = 2` costs
`and username = 'Martin';`

Purchase.dsm

Header				
1	2	3	4	5
Feb 11, 9:55	Feb 12, 10:03			
Feb 12, 10:05	Feb 12, 10:06			
Feb 19, 9:00				
Tea	Small	Small	Large	
Small				
Martin	Martin	Omar		
Omar	Martin			

DSM vs. OLTP – Exercise

Drawbacks of using DSM for OLTP workloads

```
insert into purchase
values (6, '2025-04-09 11:03'::timestamp,
       'Small', 'Martin');
```

Tuple deconstruction
costs + writing to
multiple pages or files

Purchase.dsm

Header					
1	2	3	4	5	6
Feb 11, 9:55			Feb 12, 10:03		
Feb 12, 10:05			Feb 12, 10:06		
Feb 19, 9:00			Apr 9, 11:03		
Tea	Small	Small	Large		
Small	Small				
Martin	Martin	Omar			
Omar	Martin	Martin			

Advantages

- Reduces wasted I/O when scanning large amounts of data
 - Better cache locality when working on a single attribute
 - Better data compression
- Great for OLAP workloads!

Disadvantages

- Slow for point queries, insert, updates, deletes because of tuple construction and deconstruction, and reading/writing in multiple locations

Systems in the Wild

Row Store Databases



Column Store Databases



Systems in the Wild

Big vendors offer different database products with row stores, column stores, and hybrid storage models



Newer OLAP systems and file formats with hybrid storage models



Apache Parquet is an open-source file format, not a full database system

Hybrid Storage Model

Of course, it's not as simple as just NSM vs. DSM...

People, industry and academia thought about the spectrum in between, e.g. in this research paper from 2001:

Weaving Relations for Cache Performance

Anastassia Ailamaki[‡]
Carnegie Mellon University
natassa@cs.cmu.edu

David J. DeWitt
Univ. of Wisconsin-Madison
dewitt@cs.wisc.edu

Mark D. Hill
Univ. of Wisconsin-Madison
markhill@cs.wisc.edu

Marios Skounakis
Univ. of Wisconsin-Madison
marios@cs.wisc.edu

Abstract

Relational database systems have traditionally optimized for I/O performance and organized records sequentially on disk pages using the N-ary Storage Model (NSM) (a.k.a., slotted pages). Recent research, however, indicates that cache utilization and performance is becoming increasingly important on modern platforms. In this paper, we first demonstrate that in-page data placement is the key to high cache performance and that NSM exhibits low cache utilization on modern platforms. Next, we propose a new data organization model called PAX (Partition Attributes Across), that significantly improves cache performance by grouping together all values of each attribute within each page. Because PAX only affects layout inside the pages, it

tremendous additional time to join the participating sub-relations together. Except for Sybase-IQ [33], today's relational DBMSs use NSM for general-purpose data placement [20][29][32].

Recent research has demonstrated that modern database workloads, such as decision support systems and spatial applications, are often bound by delays related to the processor and the memory subsystem rather than I/O [20][5][26]. When running commercial database systems on a modern processor, data requests that miss in the cache hierarchy (i.e., requests for data that are not found in any of the caches and are transferred from main memory) are a key memory bottleneck [1]. In addition, only a fraction of

Partition Attributes Across (PAX)

Research Paper:

DSM good for I/O, but poor tuple reconstruction cost

NSM poor I/O, but no tuple reconstruction overhead

→ Combine both, call it PAX

typically not the same size. Loading the cache with useless data (a) wastes bandwidth, (b) pollutes the cache, and (c) possibly forces replacement of information that may be needed in the future, incurring even more delays. The challenge is to repair NSM's cache behavior without compromising its advantages over DSM.

This paper introduces and evaluates **Partition Attributes Across (PAX)**, a new layout for data records that combines the best of the two worlds and exhibits performance superior to both placement schemes by eliminating unnecessary accesses to main memory. For a given relation, PAX stores the same data on each page as NSM. *Within* each page, however, PAX groups all the values of a particular attribute together on a minipage. During a

PAX Idea

PAX = Partition table by row-wise, columnar storage within partition

Product.data

Header	
Small, 85	Tea, 100
Large, 100	Filter, 110
Fancy, 125	Espresso, 35
Cortado, 35	Mocha, 45
Jasmin, 30	Cola, 25

Partition 1

Partition 2

Partition 3

NSM tuple storage



Product.pax

Header	
Small, Tea, Large, Filter	
85, 100, 100, 110	
Fancy, Espresso, Cortado	
Mocha	125, 35, 30, 25
Jasmin, Cola	30, 25

DSM tuple storage
per partition

PAX Adoption (Examples)



Product.duckdb

Header	
Small, Tea, Large, Filter	
85, 100, 100, 110	
Fancy, Espresso, Cortado	
Mocha	125, 35, 30, 25
Jasmin, Cola	30, 25

1 file split into
partitions using DSM
tuple storage



Product1.data

Header
Small, Tea, Large, Filter
85, 100, 100, 110

Product2.data

Header
Fancy, Espresso, Cortado
Mocha 125, 35, 30, 25

Product3.data

Header
Jasmin, Cola 30, 25

1 or more files with
1 partition each using
DSM tuple storage



Product1.parquet

Header
Small, Tea, Large, Filter
85, 100, 100, 110
Fancy, Espresso, Cortado
Mocha 125, 35, 30, 25
Jasmin, Cola 30, 25

Product2.parquet

Header
Small, Tea, Large, Filter
85, 100, 100, 110
Fancy, Espresso, Cortado
Mocha 125, 35, 30, 25
Jasmin, Cola 30, 25

Product3.parquet

Header
Small, Tea, Large, Filter
85, 100, 100, 110
Fancy, Espresso, Cortado
Mocha 125, 35, 30, 25
Jasmin, Cola 30, 25

Product4.parquet

Header
Small, Tea, Large, Filter
85, 100, 100, 110
Fancy, Espresso, Cortado
Mocha 125, 35, 30, 25
Jasmin, Cola 30, 25

1 or more files with 1
or more partitions each
using DSM tuple
storage

PAX Summary

Advantages

- Better I/O than NSM
 - Better tuple reconstruction than pure DSM, because attributes are “closer” together, e.g., on the same memory page as proposed in the original paper
 - Many practical benefits, e.g., inserts only affect the last partition = one file write
- PAX used in almost all commercial OLAP database systems

Disadvantages

- Tradeoff: pure columns stores better in special scenarios, e.g., specific benchmarks

Recap Over, but Who's the Winner?



OLTP vs. OLAP?

Need both!

NSM vs. DSM / PAX?

(Row store database vs. column store database)

No system that can do both!

Agenda

Catalog

- Content
- Storage & format

CSV, the input side

Data

- Binary format (row-wise, column-wise, mixed)
- Partitioning
- Statistics & pruning

Exercise 2 Preview

Catalog storage

```
public void createTable(String tableName, List<ColumnSpec> columns) { /* ... */ }

public void copyFile(String tableName, String csvFilePath) { /* ... */ }

public List<Object[]> select(String tableName, String columnName,
                             Comparison comparison, Object constant) { /* ... */ }
```

- CSV parsing
- Data storage
- **Partition statistics**

Partition pruning

Benchmarking Column Statistics for Analytical Query Pruning

Alexander Pedersen

altp@itu.dk

IT University of Copenhagen

Ismail Oukid

ismail.oukid@snowflake.com

Snowflake

Ehsan Yousefzadeh-Asl-Miandoab

ehyo@itu.dk

IT University of Copenhagen

Martin Hentschel

mhent@itu.dk

IT University of Copenhagen

Abstract

Min/max indexes are the most widely used statistics for partition pruning in analytical systems, but they miss pruning opportunities when partitions contain outliers or wide value ranges. Richer statistics, including multiple min/max indexes per partition, Bloom filters, and dictionaries, can improve pruning effectiveness at the cost of additional metadata, yet no study quantifies when this overhead is justified. We present a benchmark for evaluating column statistics for partition pruning, implemented in DuckDB, and assess pruning effectiveness and query runtimes across metadata budgets of 10× to 1000× the size of standard min/max indexes. Our results show that richer statistics yield gains for unsorted data, data with outliers, and low-cardinality columns. We derive initial guidelines for selecting column statistics based on data characteristics at load time.

The most common statistics used for partition pruning are the minimum and maximum values of columns within a partition, commonly referred to as min/max indexes or zone maps [11]. They are widely used in analytical data systems, including Snowflake [7] and DuckDB [16]. They are effective for pruning while requiring only a small amount of metadata. However, their pruning power is sometimes limited. In our measurements in Snowflake, we found that 45.5% of all scanned partitions could be pruned because they contain no qualifying tuples. 18.8% of all scanned partitions could be pruned with better metadata for equality predicates and IN lists.¹

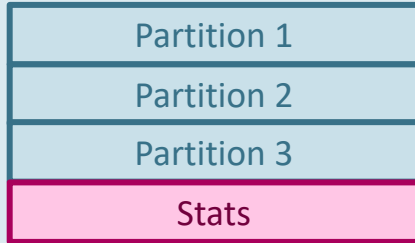
In addition to min/max indexes, richer statistics are available. Simple examples include multiple min/max indexes per column per partition (called *min/max clusters* in this paper), Bloom filters, and dictionaries. While these may offer better pruning, they also require substantially more metadata than standard min/max indexes. However, there is currently no study that quantifies when the additional metadata overhead is justified. Furthermore, there is no guidance on selecting statistics based on data characteristics during data loading. This paper aims to fill this gap by benchmarking column statistics

Data and Stats Storage in Analytical Systems



DuckDB

Data.duckdb

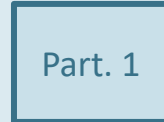


snowflake

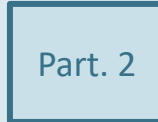
Stats.meta



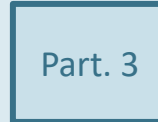
Data1.fdn



Data2.fdn

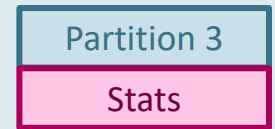
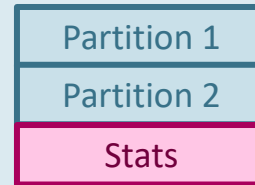


Data3.fdn



Parquet

Data1.parquet **Data2.parquet**



Data in Files

Table rows stored as tuples in files

- Header contains metadata like number of tuples, start byte of first tuple, checksums, and more
- Tuples can be of different lengths (for simplicity all same length = width below)

Product (ProductName, Price)

ProductName	Price
Small	85
Tea	100
Large	100
Filter	110
Fancy	125



Product.data

Header	
Small, 85	Tea, 100
Large, 100	Filter, 110
Fancy, 125	

Partitioning into Partitions

Database may partition the data

- Also called “row group”
- Stored as a single file or as multiple files

Think about the partition size in your engine.
Make it configurable!
What is the tradeoff?

10 tuples now for a
better example

Product.data

Header	
Small, 85	Tea, 100
Large, 100	Filter, 110
Fancy, 125	Espresso, 35
Cortado, 35	Mocha, 45
Jasmin, 30	Cola, 25



partition into
groups of rows

Product.data

Header	
Small, 85	Tea, 100
Large, 100	Filter, 110
Fancy, 125	Espresso, 35
Cortado, 35	Mocha, 45
Jasmin, 30	Cola, 25

Partition 1

Partition 2

Partition 3

Collecting Statistics

Partitioning during insertion of data

- Possibility to collect statistics: min/max indexes
- Stored e.g., in footer of the file (Apache Parquet file format)

Product.data

Header	
Small, 85	Tea, 100
Large, 100	Filter, 110
Fancy, 125	Espresso, 35
Cortado, 35	Mocha, 45
Jasmin, 30	Cola, 25

Min/max indexes per
column for each partition

Name: Filter – Tea

Price: 85 – 110

Name: Cortado – Mocha

Price: 35 – 125

Name: Cola – Jasmin

Price: 25 – 30

Product.data

Header	
Small, 85	Tea, 100
Large, 100	Filter, 110
Fancy, 125	Espresso, 35
Cortado, 35	Mocha, 45
Jasmin, 30	Cola, 25
Stats	



Pruning at Compile Time

Query optimizer decides which partitions to skip based on query predicate

- This is called “pruning”
- Usually happens at compile time
- Pruning in our engine for operators =, <, >

```
select * from product
where productname = 'Cola';
```

Predicate is
productname = 'Cola'

Optimizer skips partitions 1 and 2
because min(productname) of these
partitions > 'Cola' (alphabetically)

Stats

Partition 1

Name: Filter – Tea

Price: 85 – 110

Partition 2

Name: Cortado – Mocha

Price: 35 – 125

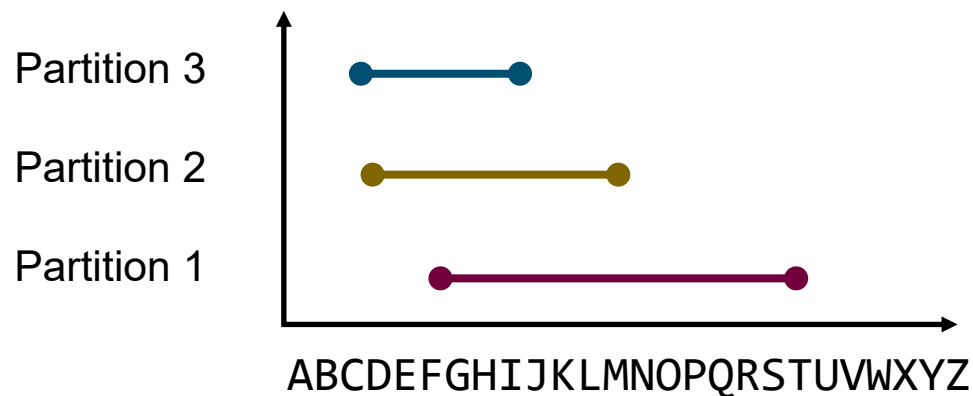
Partition 3

Name: Cola – Jasmin

Price: 25 – 30

Visualization of Min/Max Indexes

Each line represents the min and max values for column 'name' per partition



Stats

Partition 1

Name: Filter – Tea

Price: 85 – 110

Partition 2

Name: Cortado – Mocha

Price: 35 – 125

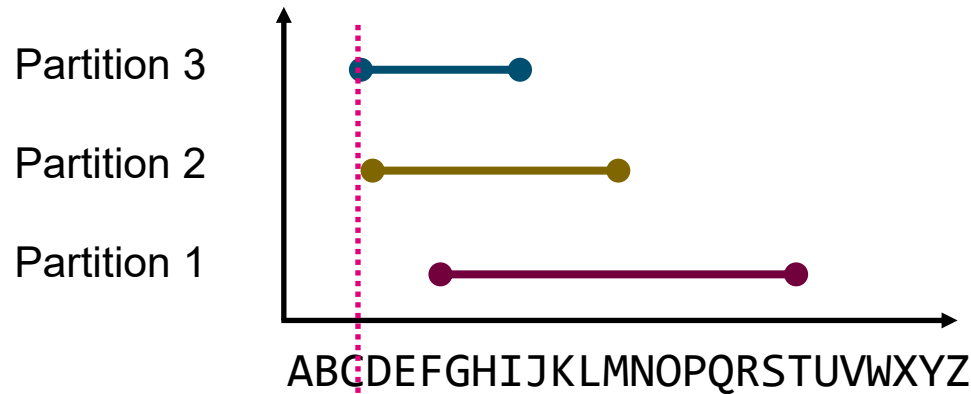
Partition 3

Name: Cola – Jasmin

Price: 25 – 30

Visualization of Query Predicate 'Cola'

Query predicate is a vertical line in the chart: if it cuts min/max $\rightarrow$ cannot prune



Predicate

productname = 'Cola'

$\rightarrow$ Cannot prune partition 3

$\rightarrow$ But prune partitions 1 and 2

Stats

Partition 1

Name: Filter – Tea

Price: 85 – 110

Partition 2

Name: Cortado – Mocha

Price: 35 – 125

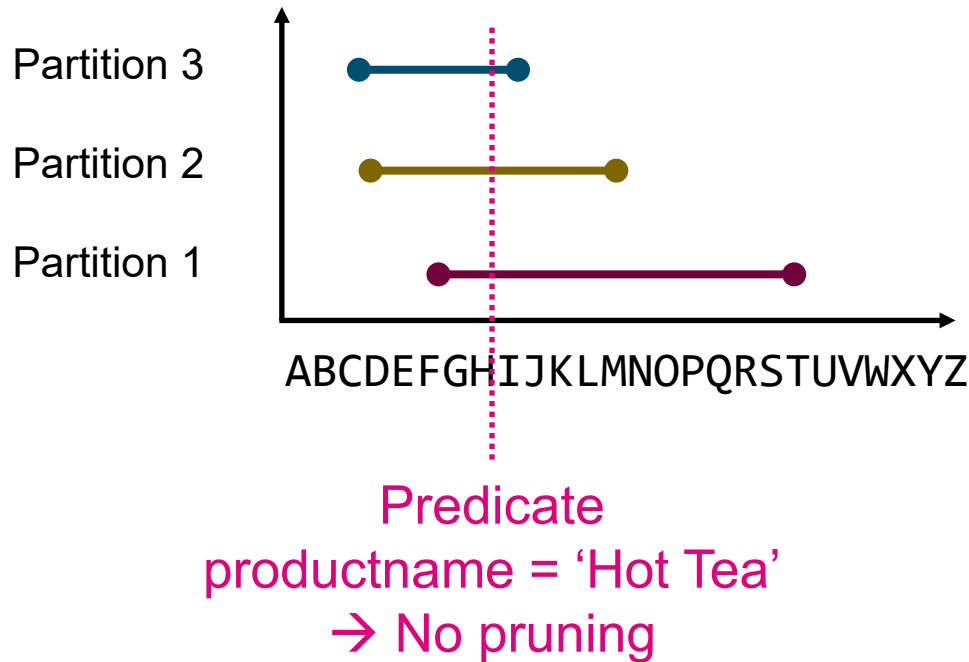
Partition 3

Name: Cola – Jasmin

Price: 25 – 30

Visualization of Query Predicate 'Hot Tea'

Query predicate is a vertical line in the chart: if it cuts min/max $\rightarrow$ cannot prune



Stats

Partition 1

Name: Filter – Tea

Price: 85 – 110

Partition 2

Name: Cortado – Mocha

Price: 35 – 125

Partition 3

Name: Cola – Jasmin

Price: 25 – 30

Sorting the Table by ProductName

Sorted by ProductName:

Product.data

Header	
Cola, 25	Cortado, 35
Espresso, 35	Fancy, 125
Filter, 110	Jasmin, 30
Large, 100	Mocha, 45
Small, 85	Tea, 100

Min/max indexes per
column for each partition

Name: Cola – Fancy
Price: 25 – 125

Name: Filter – Mocha
Price: 30 – 110

Name: Small – Tea
Price: 85 – 100

Different indexes because
now data is sorted by
product name

Stats

Partition 1

Name: Cola – Fancy
Price: 25 – 125

Partition 2

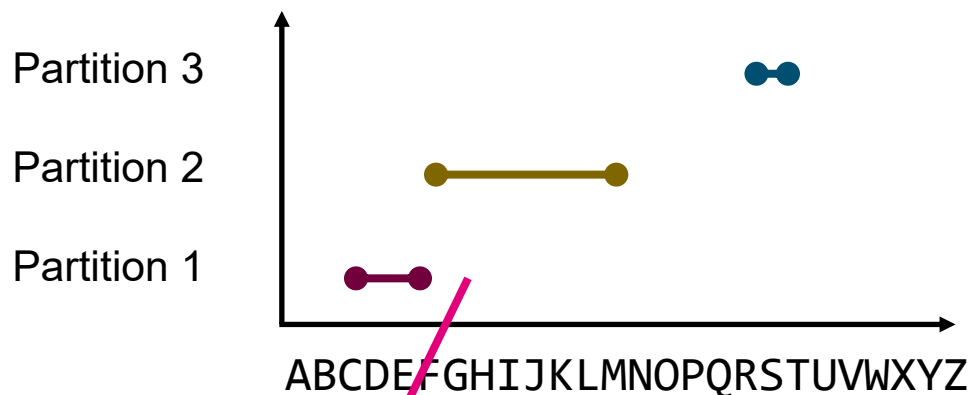
Name: Filter – Mocha
Price: 30 – 110

Partition 3

Name: Small – Tea
Price: 85 – 100

Visualization of Min/Max indexes

Sorted by ProductName:



No overlapping min/max indexes →
Good pruning potential for any
predicate on productname

Stats

Partition 1

Name: Cola – Fancy

Price: 25 – 125

Partition 2

Name: Filter – Mocha

Price: 30 – 110

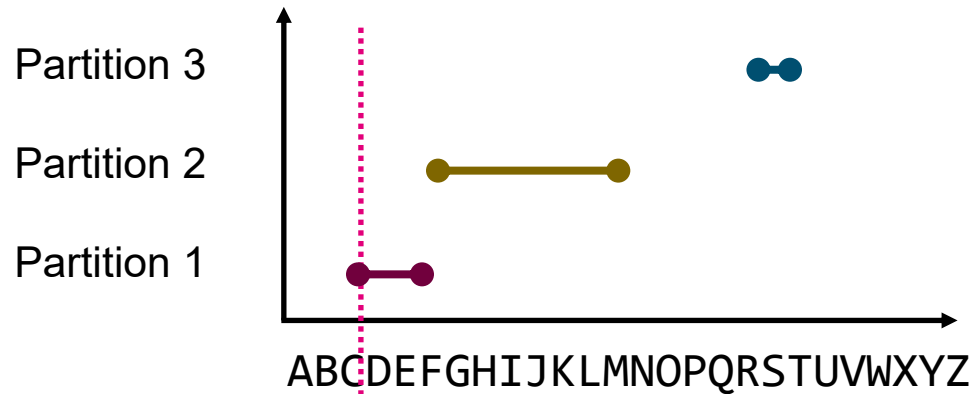
Partition 3

Name: Small – Tea

Price: 85 – 100

Visualization of Query Predicate 'Cola'

Perfect pruning for 'Cola'



Predicate

productname = 'Cola'

→ Prune partitions 2 and 3

Stats

Partition 1

Name: Filter – Tea

Price: 85 – 110

Partition 2

Name: Cortado – Mocha

Price: 35 – 125

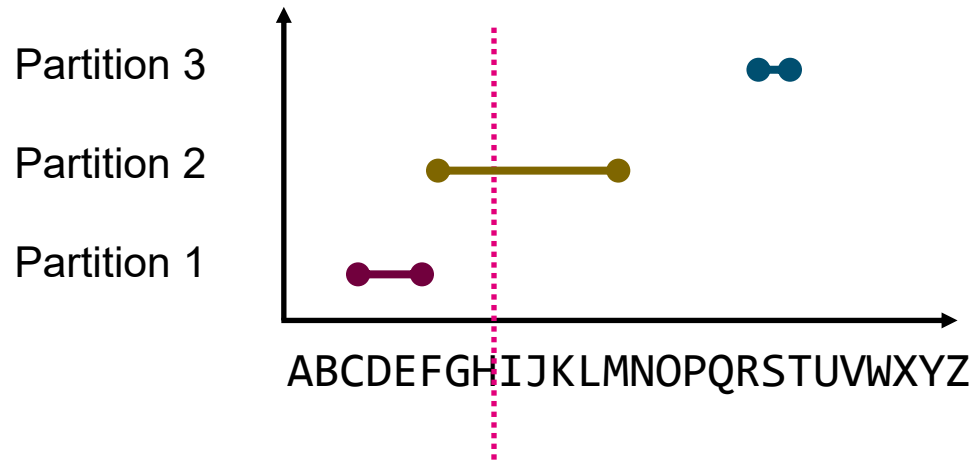
Partition 3

Name: Cola – Jasmin

Price: 25 – 30

Visualization of Query Predicate 'Hot Tea'

Perfect pruning for 'Hot Tea'



Predicate
productname = 'Hot Tea'
→ Prune partitions 1 and 3

Stats

Partition 1

Name: Filter – Tea

Price: 85 – 110

Partition 2

Name: Cortado – Mocha

Price: 35 – 125

Partition 3

Name: Cola – Jasmin

Price: 25 – 30

Summary: Min/Max Indexes

Most-widely used statistics for summarizing partitions

- DuckDB, Snowflake, Parquet, Oracle, ...
- 99.4% of partitions pruned in Snowflake*

However: 45.5% of all *scanned* partitions could be pruned because they contain no qualifying tuples

- 18.8% of all scanned partitions could be pruned with better stats for equality predicates and IN lists**

* Andreas Zimmerer, Damien Dam, Jan Kossmann, Juliane Waack, Ismail Oukid, and Andreas Kipf. *Pruning in Snowflake: Working Smarter, Not Harder*. SIGMOD 2025

**Measurements for our paper

Alexander Pedersen
altp@itu.dk
IT University of Copenhagen

Ismail Oukid
ismail.oukid@snowflake.com
Snowflake

Ehsan Yousefzadeh-Asl-Miandoab
ehyo@itu.dk
IT University of Copenhagen

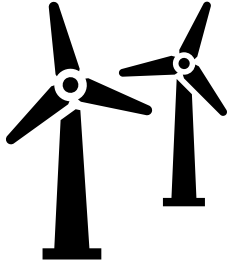
Martin Hentschel
mhent@itu.dk
IT University of Copenhagen

Study pruning efficiency of different statistics under varying workloads, data distributions, cardinalities.

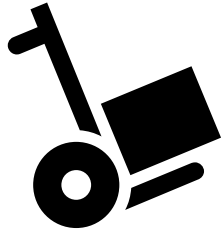
Contributions:

- Definition of a micro benchmark based on real-world use cases
- Results that larger metadata sizes (10-100x) can be beneficial
- Initial set of guidelines for selecting stats on insert

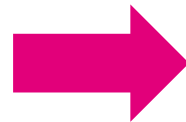
Micro Benchmark Use Cases



Sensor networks sometimes send outdated data, leading to partitions with “wide” min/max ranges.



During data migrations, null values can be misrepresented in the target database, e.g. dates with default values such as 1970-01-01 or 9999-12-31.



Outliers

Micro Benchmark Design

SensorData (MeasurementTime: bigint, SensorID: bigint)

Data

- MeasurementTime: random 64-bit ints resembling epoch ms between Jan 1, 2026 and Jan 31, 2026
- SensorID: random 64-bit ints, restricted to cardinality

Characteristics

- 1 billion rows = 20 GB
- 8,192 rows per partitions = 122,071 partitions in total
- MeasurementTime: sorted, unsorted, outliers in every 10,000th row
- SensorID cardinality: 100 to 100,000 distinct values

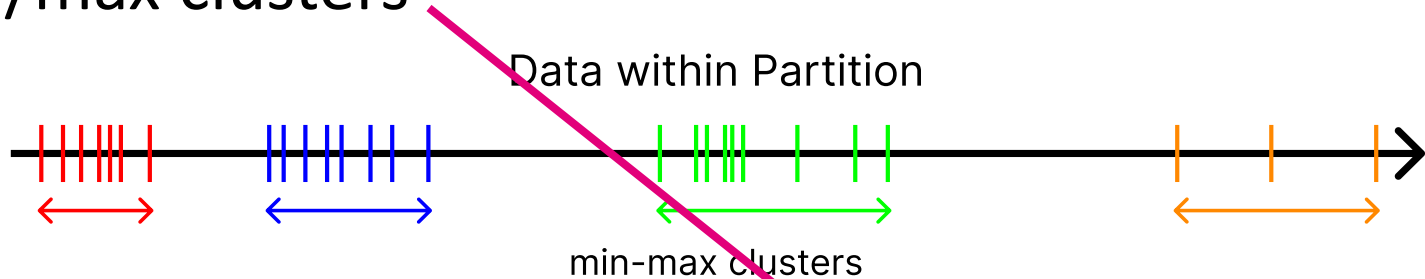
Queries

- by MeasurementTime (point and range) and SensorID (point)

Analyzed Statistics

Min/max indexes (baseline)

Min/max clusters

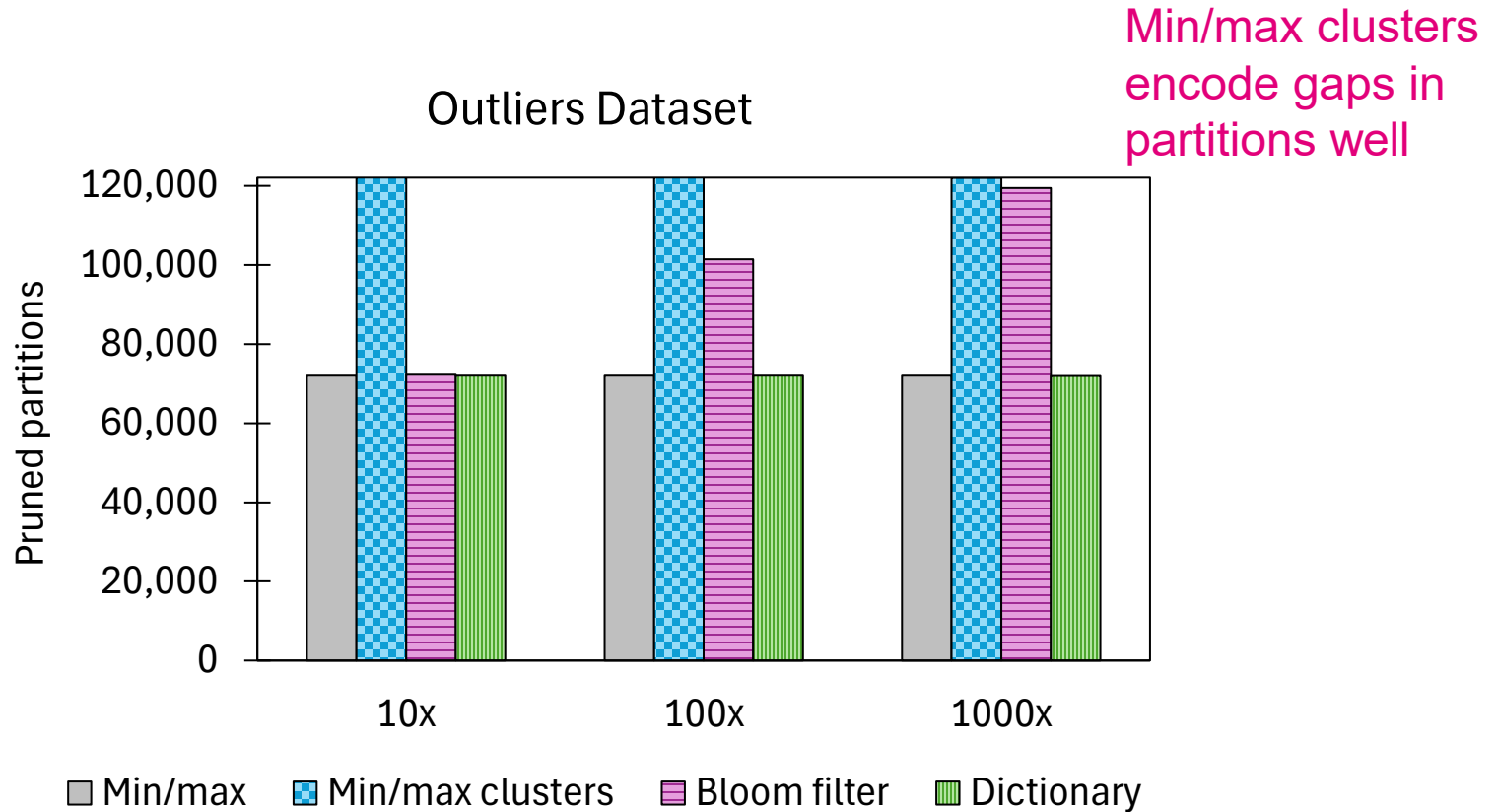


Dictionaries = hash sets

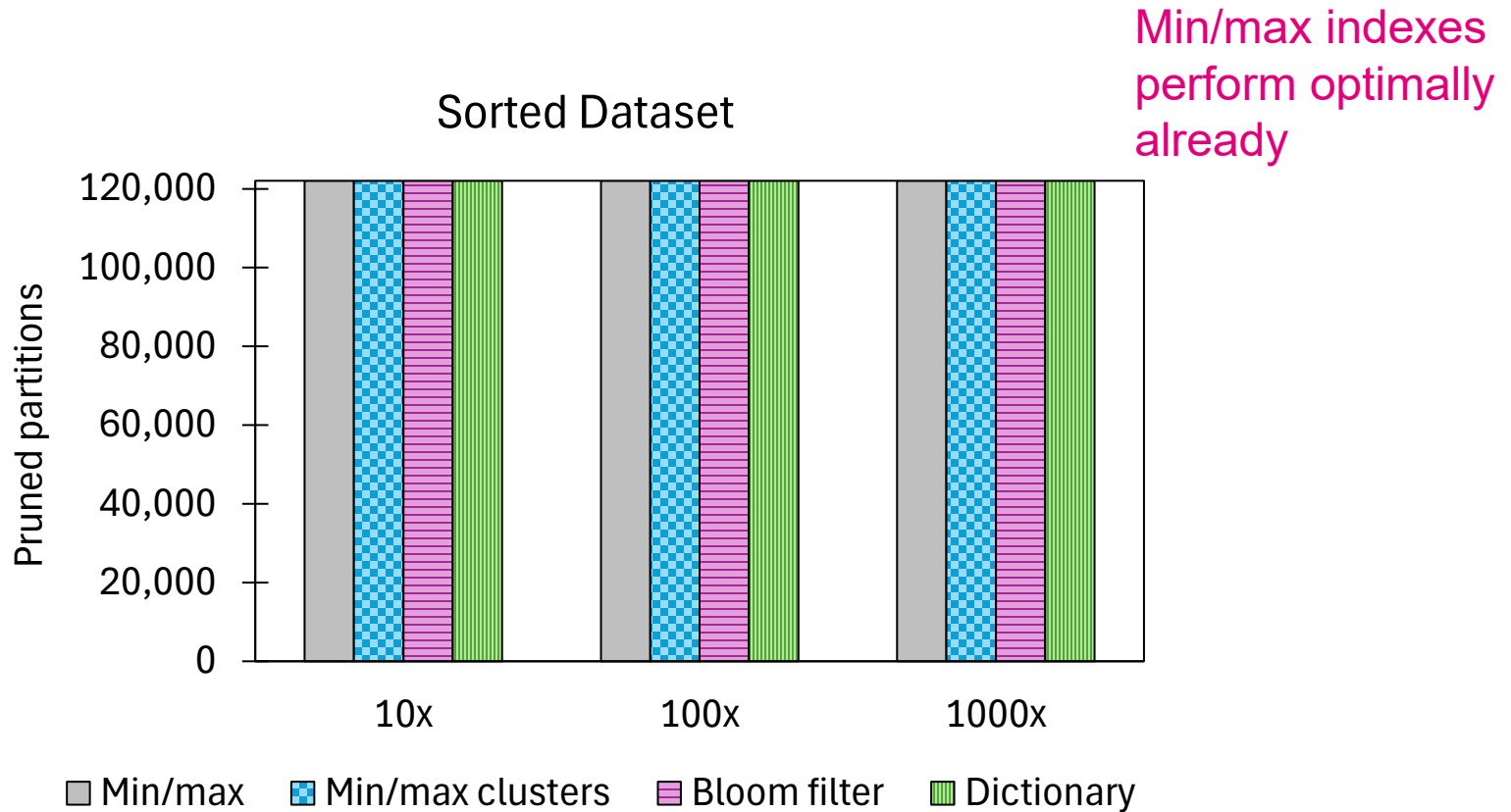
Bloom filters

Always “on top” of
baseline

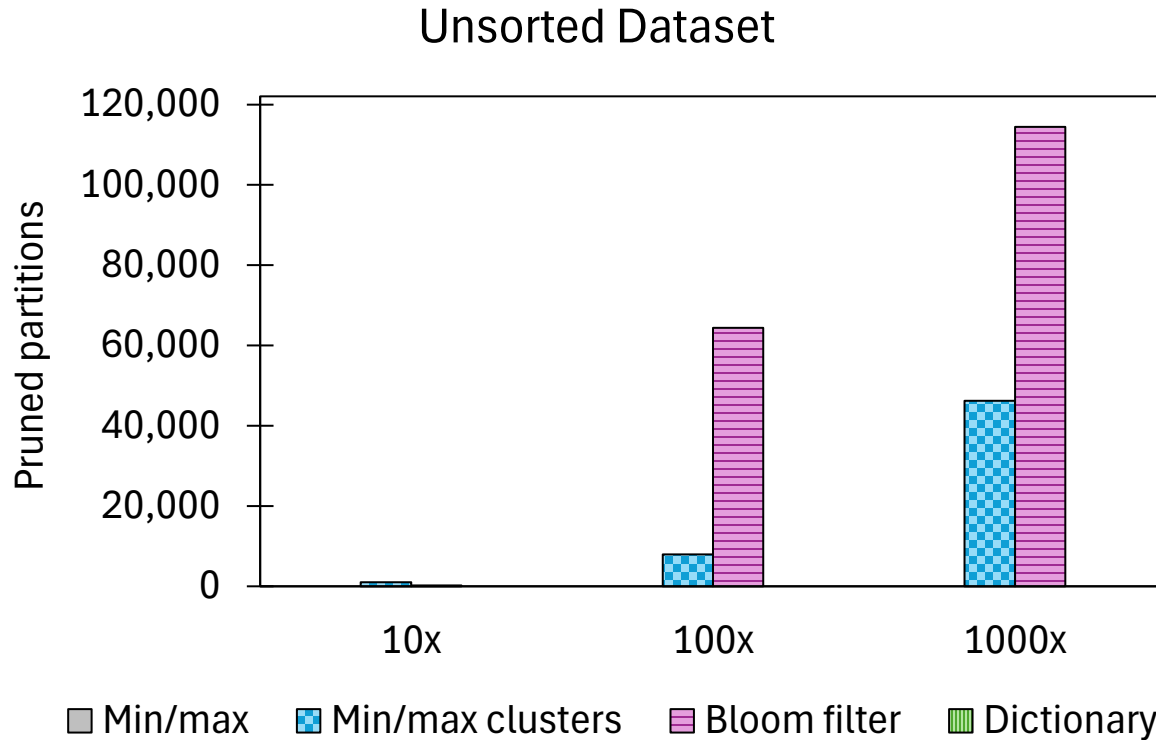
Experiment: Sorted Data with Outliers



Experiment: Sorted Data



Experiment: Unsorted Data, High Cardinality



Many random values best encoded by (large) Bloom filters

Summary and Trade-Offs

Catalog: stores table metadata, list of partitions and files, partition stats

One file vs. many
files vs. DB/KV

JSON vs. AVRO/Protobuf
vs. own format

Stats inside vs.
stats outside

Data storage

N-ary vs. columnar
vs. PAX

One file per DB vs. one file per
table vs. many files per table

Statistics

Partition size:
pruning power vs.
metadata size

Min/max only
vs. other stats